



**PROJECT NAME: VULNERABILITY
MANAGEMENT DATABASE FOR OPEN-SOURCE
SOFTWARE (VMDB-OSS)**

**BACHELOR OF BUSINESS ADMINISTRATION
(Hons.) IN ARTIFICIAL INTELLIGENCE
VERSION – 1.1**

Project Under: Database Management System, Mini Project Submission

*Created & Managed by
Ashwin Chhawaniya
25030422028
Section – A, BBA (Hons.) AI
Symbiosis Artificial Intelligence Institute*

*Submitted to
Dr. Pradnya Patil Ma'am
Assistant Professor
Symbiosis Artificial Intelligence Institute*

VMDB - OSS

VMDB_OSS: Vulnerability Management Database for Open Source Software

The Open-Source Vulnerability Management Database Designed for Enhanced Security and Integrity

A Comprehensive Project Design Document

Author: Project Maintainer Date: November 13, 2025 Version: 1.2 (High-Density Edition)

Table of Contents

Section	Title	Description
1.0	Project Introduction	High-level vision, problem statement, goals, and scope.
1.1	Vision & Motivation	Why this project exists and the need it fills in modern security.
1.2	Problem Statement	The specific business and technical challenges this database solves.
1.3	Project Purpose & Goals	The primary objectives and desired outcomes, in detail.
1.4	Scope & Constraints	A clear boundary of what is included (and excluded).
1.5	Intended Audience & Personas	Who this project is built for, with specific personas.
1.6	Document Overview	A guide to reading this document.
2.0	High-Level Features	The four key pillars of the VMDB_OSS project.
2.1	Normalized Relational Schema	The core database design principles (3NF) and their benefits.
2.2	Realistic & Coherent Data	The value of the sample data as a teaching and testing tool.
2.3	Analytics-Ready Views	The "semantic layer" for simple reporting and BI.
2.4	Operational Stored Procedures	The "API layer" for security, consistency, and operations.
3.0	Database Schema Deep Dive	A detailed analysis of the complete data model.
3.1	Conceptual Model & ERD	A high-level overview of entity relationships and cardinality.
3.2	Schema Overview	A list of all tables and their core purpose.
3.3	Table-by-Table Analysis	A deep, column-by-column breakdown of each table.
4.0	Data Model & Sample Data	A walkthrough of the sample data to show coherency.
4.1	Rationale for Sample Data	Why this specific data was chosen.
4.2	Data Coherency & Storytelling	How the data links together to model real scenarios (2 examples).
4.3	Sample Data Walkthrough	A narrative summary of the inserted data.
5.0	Database Views (The Analytics Layer)	A deep dive into each VIEW and its purpose.
5.1	Purpose of the View Layer	Why views are used for abstraction, security, and simplicity.

Section	Title	Description
5.2	View-by-View Analysis	A detailed, tabular breakdown of all 6 views and their stakeholder questions.
6.0	Stored Procedures (The Operational Layer)	A deep dive into each <code>PROCEDURE</code> and <code>FUNCTION</code> .
6.1	Purpose of Stored Procedures	Why procedures are used for encapsulation, security, and performance.
6.2	Procedure Analysis	A detailed, tabular breakdown of the included procedures and functions.
7.0	Example Workflows & Use Cases	Practical, step-by-step scenarios for using the DB.
7.1	Use Case 1: Daily Scan	How to ingest new vulnerability data.
7.2	Use Case 2: Remediation	How to track a patch from "Open" to "Resolved".
7.3	Use Case 3: Reporting	How to answer management-level questions.
8.0	Deployment & Operations	Instructions for installing and running the database.
8.1	System Requirements	What is needed to run the schema.
8.2	Installation & Setup	How to deploy the database (see SQL file).
8.3	Verification & Basic Usage	Simple queries to confirm success, including a failure test.
9.0	Project Deliverables & Evaluation	A checklist for project submission or review.
9.1	Core Artifacts	The files that make up this project.
9.2	Demonstration Points	A "script" for showing off the project's features.
10.0	Future Roadmap & Extensions	Potential next steps to expand the project.
10.1	Short-Term Goals	UI, automated ingestion, and SBOM parsing.
10.2	Medium-Term Goals	Advanced scoring, automation, and integration.
10.3	Long-Term Goals	Enterprise features (RBAC, Tenancy) and deployment.
11.0	Conclusion	A summary of the project's value and contribution.

1. Project Introduction

1.1. Vision & Motivation

Vision: To provide a simple, robust, and accessible database backend for tracking open-source software vulnerabilities, enabling any developer, team, or small organization to move from reactive patching to proactive security management.

Motivation: Open-source software (OSS) is the backbone of modern technology. It is no longer a niche component but the fundamental building block of enterprise applications, cloud infrastructure, and AI models. This ubiquity introduces a critical challenge: **Software Supply Chain Security**. A modern application is an assembly of hundreds of third-party packages, and a single vulnerability in a seemingly minor downstream package can create a critical security hole in thousands of upstream projects (e.g., Log4Shell).

This project is motivated by the fact that before an organization can *manage* its risk, it must first *measure* it. Many commercial tools exist (Snyk, Dependabot, JFrog Xray), but they can be expensive, complex, or "black box" solutions. A "textbook-clean" yet practical database schema provides a powerful, transparent alternative for:

- **Learning:** This schema provides a clear, tangible case study for students to visualize and query the complex, abstract relationships between projects, packages, and CVEs. It transforms theoretical database design and cybersecurity concepts into a practical, hands-on model that can be immediately explored with SQL.
- **Small Orgs:** Teams with limited budgets, who are currently relying on disparate spreadsheets and manual tracking, are empowered to build lightweight, custom internal tools. This schema provides a solid, reliable foundation, allowing them to centralize their data and automate reporting without the cost of enterprise-grade commercial software.
- **Maintainers:** Open-source maintainers, who are often inundated with security alerts, can use this schema to track dependencies and vulnerabilities for their own projects. It provides a free, simple, and self-hostable alternative to heavy-handed enterprise tooling, fitting their workflow and needs without unnecessary overhead.

This project is motivated by the need for a transparent, educational, and functional "home base" for vulnerability data—a single source of truth that is both easy to understand and powerful enough to build upon.

1.2. Problem Statement

Organizations and maintainers lack a straightforward, centralized, and reliable way to answer fundamental security questions about their open-source dependencies. This "vulnerability information gap" is not a minor inconvenience; it's a critical business risk that leads to several negative outcomes:

- **Unknown Risk:** Without a central database, it's impossible to know the organization's true risk posture at any given moment. An analyst cannot answer the simple, time-sensitive question, "Are we vulnerable to the new 'Critical' CVE that was just announced?" The answer is buried in static, outdated reports or individual developer knowledge.
- **Developer Friction:** In the absence of a central system, developers are left to track vulnerabilities in an ad-hoc way, often using inconsistent methods. This is grossly inefficient, pulls highly-paid engineers away from core feature development, and leads to frustration and burnout as they are forced to be part-time security accountants.
- **Failed Audits:** When an auditor or compliance officer asks, "Show me your documented process for tracking and remediating all 'High' severity vulnerabilities," the organization is in trouble. They cannot produce a coherent log, a remediation history, or a report, leading to compliance failures, lost certifications, or even fines.
- **Inefficient Remediation:** Teams waste time and resources. Without a prioritized, central list, multiple developers may unknowingly research the same vulnerability. Worse, high-priority, high-impact fixes (e.g., a critical RCE) may be missed entirely while developers focus on easier, low-priority fixes that appear in their local scanner first.

Without a structured database, this critical security information lives in fragmented, unreliable sources: static scanner reports, ephemeral CI/CD logs, disconnected email chains, and tribal-knowledge spreadsheets. This is inefficient, error-prone, and makes comprehensive risk reporting nearly impossible. `VMDB_OSS` provides the relational structure to solve this problem.

1.3. Project Purpose & Goals

The **primary purpose** of `VMDB_OSS` is to model the complete lifecycle of an open-source vulnerability as it relates to a portfolio of software projects. It is an "Asset and Vulnerability Management" (AVM) database specifically tailored for the complexities of Open-Source Software.

Core Goals:

1. **Model Relationships:** The schema must accurately represent the complex "graph" of software dependencies. This includes the vital many-to-many (M:N) relationship between `Projects` and `Packages` (resolved by the `Project_Packages` table) and the one-to-many (1:N) relationship between a `Package` and its list of `Vulnerabilities`. This model is the core "worldview" of the database.

2. **Track Lifecycle:** The system must enable end-to-end tracking of a vulnerability's remediation lifecycle. This is not a single flag, but a process: a vulnerability is first discovered (*Pending*), then actively worked on (*In Progress*), and finally verified as fixed (*Resolved*). The database must also capture the *evidence* of this, such as *when* the fix was applied and *what* version it was patched in.
3. **Ensure Integrity:** The database must be the source of truth, and that truth must be trustworthy. This is achieved by using database constraints to guarantee that the data is clean, accurate, and interconnected. `FOREIGN KEY` constraints are the most important tool, making it *impossible* to have "orphaned" records (e.g., a remediation for a non-existent vulnerability). `ENUM` and `NOT NULL` constraints ensure all required data is present and valid.
4. **Provide Insight:** The database must not be a "write-only" black hole. It must offer immediate value by providing pre-built `VIEW` s. These views aggregate raw, normalized data (which is efficient for storage but hard to read) into de-normalized, actionable intelligence. These views are designed to directly answer the key questions of different stakeholders (analysts, managers, developers) without complex SQL.
5. **Simplify Operations:** The database must provide a clear, safe, and simple "API" for applications to use. `STORED PROCEDURE` s are this API, encapsulating common business logic (like updating a status or adding a remediation record). This reduces code duplication in applications, improves security by preventing SQL injection, and ensures that all data writes follow the same consistent rules.

1.4. Scope & Constraints

- **In Scope:**
 - **A single, self-contained MySQL 8.0 compatible schema:** This is the primary deliverable. It includes all table definitions, constraints, and relationships necessary to model the problem domain.
 - **All DDL (`CREATE`) statements:** This includes the SQL to create all tables, views, stored procedures, and functions. The script will be complete and runnable.
 - **A representative DML (`INSERT`) script:** The project includes a coherent sample dataset (approximately 10 rows per table). This data is interconnected and realistic, making the schema immediately usable and queryable for demonstration and development.
 - **This design document:** A comprehensive `README.md` file that provides a "glass box" explanation of the schema's design, purpose, entity relationships, and use cases, as you are reading now.
- **Out of Scope (for this phase):**
 - **A User Interface (UI):** This is a *backend* project. It is designed to be the "brain" or "persistence layer" that a future web application, command-line tool, or BI dashboard would connect to. No HTML, CSS, or JavaScript is included.
 - **Data Importers:** The project does not include the Python or shell scripts that would be needed to automatically fetch data from external APIs like the NVD, GitHub Advisories, or other vulnerability feeds. It provides the *storage* for that data once it is fetched.
 - **Scanner Logic:** This database *stores the results* of a scan. It is not the scanner itself. It does not contain the logic for "trivy," "snyk-cli," or any other tool that inspects a project's dependencies.
 - **A SIEM (Security Information and Event Management):** This is not a log-shipping or real-time event correlation engine. It is a relational, stateful database for tracking assets, their known vulnerabilities, and the long-term status of remediation efforts.
 - **Advanced Access Control:** The schema includes a `Users.role` field which *enables* permissions logic, but it does not implement fine-grained, row-level security (RLS) or complex database-level `GRANT` policies.
- **Design Constraints:**
 - **Simplicity & Readability:** The schema prioritizes human-readable tables, column names, and clear, simple foreign key relationships. The goal is a schema that a student or new developer can look at and understand the business logic within minutes, without complex abstraction.

- **Portability:** The schema uses standard, cross-compatible MySQL 8.0 features. It deliberately avoids complex, version-specific, or proprietary extensions. This ensures it can run on any standard MySQL instance (local, cloud, etc.) without modification.
- **Relational Purity:** The design avoids using semi-structured data types (like `JSON`) for storing core, relational data. While `JSON` can be useful, using a pure relational model (specifically Third Normal Form, 3NF) makes the schema easier to learn, query with standard SQL, and enforce rigid data integrity.

1.5. Intended Audience & Personas

This project and its documentation are designed for four distinct audiences:

- **Database Students (Persona: "Alex"):** Alex is in a university database course. This project serves as a perfect case study for data modeling, normalization (specifically 3NF), creating complex `JOIN` queries, and writing views and procedures. It provides a real-world, tangible problem domain that is more engaging than a textbook "customers and orders" example.
- **Security Analysts (Persona: "Neha"):** Neha works at a small-to-medium business. She's tired of using error-prone spreadsheets to track vulnerabilities. This project gives her a blueprint for building a simple, custom internal vulnerability tracking system. She can deploy this schema and immediately connect it to a BI tool (like Metabase or Tableau) to build live dashboards.
- **Software Developers (Persona: "Amit"):** Amit is a maintainer of several open-source projects. He's curious about how the dependency scanners (like Dependabot) he uses daily actually *work* under the hood. This schema helps him understand the underlying data structure, and he might even use it to track dependencies and known issues for his own personal projects.
- **Instructors (Persona: "Ravi"):** Ravi teaches a database or cybersecurity course. This project is a ready-to-use lab or mini-project. He can give his students the `.sql` file and a set of business questions ("Which maintainer has the most 'Critical' pending vulnerabilities?") and have them write the corresponding SQL queries.

1.6. Document Overview

This document provides a "glass box" view of the `VMDB_OSS` database, broken down into 11 sections. It will first cover the high-level features and design pillars. It then dives deeply into the schema, analyzing each table and column's specific purpose and rationale. Following this, it will provide a deep, tabular analysis of each `VIEW` and `STORED PROCEDURE`, explaining their logic and stakeholder purpose. Finally, it will walk through practical, step-by-step use cases, provide deployment instructions, and discuss the project's future roadmap.

2. High-Level Features

`VMDB_OSS` is more than just a collection of tables; it's a small, functional system with four key feature pillars.

2.1. Normalized Relational Schema

The schema is designed in Third Normal Form (3NF), a database design standard that minimizes data redundancy and improves data integrity. This is not just an academic exercise; it has practical benefits.

- **No Redundancy (Minimization):** A package's name (e.g., `Werkzeug`) is stored *once* in the `Packages` table (ID: 1). The `Project_Packages` table references this package using its efficient 4-byte integer ID (`package_id: 1`), not by repeating the 8-byte string "Werkzeug" hundreds of times. This saves space and improves query speed.
- **Prevents Data Anomalies:** This design prevents "update anomalies." If the 'Werkzeug' package were to be renamed to 'Werkzeug-Core', a non-normalized schema would require updating every single project that uses it. In this 3NF schema, you only have to update it in *one place* (`Packages` table, row 1), and all related records (projects, vulnerabilities) automatically see the change.

- **Data Integrity Guarantee:** `FOREIGN KEY` constraints create an enforced "web" of relationships. It is *impossible* for the database to contain a remediation record for a `vuln_id` that doesn't exist. It is *impossible* to assign a project to a `maintainer_id` that isn't in the `Users` table. This integrity is the database's primary guarantee of data quality.
- **Clarity and Separation of Concerns:** The design is clear and logical. `Projects` stores information about projects. `Vulnerabilities` stores information about vulnerabilities. The `Project_Packages` table exists *only* to link them. This separation makes the schema easy to understand, maintain, and extend.

2.2. Realistic & Coherent Data

The provided sample data (`INSERT` statements) is not random. It's a curated dataset that tells a story, making the database immediately useful for experimentation, learning, and development.

- **Real-world Examples:** The data uses well-known projects (Flask, React, TensorFlow) and real (or realistic) packages (Lodash, NumPy, Express) and CVE identifiers. This makes the data relatable and provides immediate context for anyone in the tech industry.
- **Data Coherency:** The data is deeply interconnected. The `Flask` project (ID 1) is linked to `Werkzeug` (ID 1). `CVE-2024-23334` (ID 1) is linked to `Werkzeug`. This allows for immediate, meaningful `JOIN` queries that show Flask is vulnerable. Without this coherency, queries would return empty results and the schema would feel broken.
- **A Complete Lifecycle Example:** The `Angular` project (ID 8) uses `Lodash` (ID 8) at version `4.17.20`. A vulnerability (`CVE-2024-16002`, ID 9) is known for `Lodash` (package_id 8). A `Remediation` (ID 9) shows this was `Resolved` for the Angular project by updating to `4.17.21`. This single example demonstrates a complete, end-to-end resolved lifecycle, which is invaluable for testing and understanding.

2.3. Analytics-Ready Views

The `VIEW`s act as a pre-built "analytics layer" or "semantic layer." They abstract the complexity of `JOIN`s away from the end-user, transforming raw data into business intelligence.

- **Abstraction and Simplicity:** A manager (Persona: "Priya") doesn't need to know *how* to join seven tables, use `LEFT JOIN`s, and `COALESCE` null values. She can simply `SELECT * FROM vw_project_vulnerabilities` to get the answer she needs. The view hides the *how* and provides the *what*.
- **Semantic Layer:** The views "translate" the raw, normalized data (e.g., `pkg_id: 1`, `user_id: 3`) into human-readable information (e.g., `package_name: 'Werkzeug'`, `maintainer_name: 'Amit Shah'`). This layer is what a Business Intelligence (BI) tool (like Tableau, Metabase, or Power BI) would connect to.
- **Performance & Security:** Views can be used to restrict access to data. A user can be given `SELECT` permissions on a view (e.g., `vw_critical_vulnerabilities`) without having any permissions on the underlying `Vulnerabilities` table. This is a powerful, simple security control.

2.4. Operational Stored Procedures

The `STORED PROCEDURE`s act as a simple, safe, and auditable "API" for the database. They are the *verbs* (actions) that complement the tables' *nouns* (entities).

- **Security (Preventing SQL Injection):** This is the most critical reason to use them. Instead of an application building a dynamic SQL string like `INSERT INTO ... VALUES ('" + userInput + "')` (which is a massive security hole), it safely calls a pre-defined procedure: `CALL AddRemediation(1, 2, '3.0.1', 'Resolved')`. The database engine handles the parameterization safely, neutralizing the threat of SQL injection.

- **Consistency and Business Logic:** Business rules are enforced centrally. The `AddRemediation` procedure *always* sets the `date_applied` to `CURDATE()`. This logic lives in *one place* (the database), not scattered across 10 different microservices or client applications. This ensures no one can "forget" to set the date or insert an invalid one.
- **Performance and Maintenance:** Stored procedures can have their execution plans compiled and cached by the database engine, leading to better performance for frequently executed, complex write operations. Furthermore, if your business logic changes (e.g., you need to log to the `Audit_Log` every time a remediation is added), you change *one procedure*, and all applications that call it are instantly updated.

3. Database Schema Deep Dive

3.1. Conceptual Model & ERD

The conceptual model is based on a set of core entities (`Users` , `Projects` , `Packages` , `Vulnerabilities`) and the relationships between them. These relationships define the business logic of the vulnerability management process.

Conceptual Model Breakdown (Tabular Form):

Entity 1	Relationship	Entity 2	Cardinality (E1 to E2)	Business Rule / Description
<code>Users</code>	Maintains	<code>Projects</code>	One-to-Many (1:N)	A User (Maintainer) can be responsible for many Projects. A single Project has exactly one Maintainer, creating accountability. This is enforced by the <code>maintainer_id</code> Foreign Key in the <code>Projects</code> table.
<code>Users</code>	Performs	<code>Audit_Log</code>	One-to-Many (1:N)	A User can perform many Actions that are logged. Each Audit Log entry is tied to exactly one User. This creates the "who" for the audit trail.
<code>Projects</code>	Uses	<code>Packages</code>	Many-to-Many (M:N)	A Project can use many Packages. A Package can be used by many Projects. This complex relationship is the core of the schema.
<code>Project_Packages</code>	<i>Resolves</i>	<i>(Projects/Packages)</i>	(Junction Table)	This table <i>resolves</i> the M:N relationship. A row in this table represents a <i>single link</i> : "Project X uses Package Y, at version Z". This is the "Software Bill of Materials" (SBOM) table.
<code>Packages</code>	Has	<code>Vulnerabilities</code>	One-to-Many (1:N)	A Package can have zero, one, or many Vulnerabilities discovered against it. Each Vulnerability is tied to exactly one Package. This links a

Entity 1	Relationship	Entity 2	Cardinality (E1 to E2)	Business Rule / Description
				general problem (CVE) to a specific asset (Package).
Projects	Has	Vulnerabilities	Many-to-Many (M:N)	A Project can be affected by many Vulnerabilities (via its packages). A Vulnerability can affect many Projects (any project using the package). This M:N relationship is the <i>problem</i> we are tracking.
Remediations	<i>Resolves</i>	<i>(Projects/Vulnerabilities)</i>	(Junction Table)	This table <i>resolves</i> the M:N relationship above. A row in this table represents a <i>single fix</i> : "The fix for Vulnerability Y on Project X has status 'Pending'". This is the "Action" or "Ticket" table.

3.2. Schema Overview

The database consists of 7 core tables, divided into Entities and Relations:

Entity Tables (The "Nouns"):

1. **Users** : Identity store for all system users (Admins, Analysts, Maintainers).
2. **Projects** : The software projects being tracked (e.g., 'Flask', 'React').
3. **Packages** : The open-source components ('Werkzeug', 'Lodash').
4. **Vulnerabilities** : The specific CVEs or security flaws ('CVE-2024-23334').

Relational / Junction Tables (The "Verbs"):

5. **Project_Packages** : The "Usage" link. It maps which version of a package a project uses.
6. **Remediations** : The "Fix" link. It logs fixes applied for a specific vulnerability in a specific project.
7. **Audit_Log** : The "Action" link. A high-level log of user actions within the system.

3.3. Table-by-Table Analysis

3.3.1. Users

- **Purpose:** This table manages identity, authentication, and high-level roles. It is the "who" of the database.
- **Auditor's View:** An auditor would query this table to ensure proper separation of duties (e.g., "Are all 'Maintainers' distinct from 'Admins'?").

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
<code>user_id</code>	<code>INT</code>	<code>AUTO_INCREMENT</code> , <code>PRIMARY KEY</code>	Unique numeric identifier for the user. Used as the Foreign Key in other tables.
<code>full_name</code>	<code>VARCHAR(100)</code>	<code>NOT NULL</code>	The user's full name. <code>VARCHAR(100)</code> is sufficient for most names. <code>NOT NULL</code> as an account must have a name.

Column Name	Data Type	Constraints	Description/Purpose
email	VARCHAR(100)	UNIQUE , NOT NULL	The user's email, used for login and uniqueness. UNIQUE is critical to prevent duplicate accounts.
role	ENUM(...)	DEFAULT 'Maintainer'	Controlled list: 'Admin', 'Maintainer', 'Analyst'. ENUM is a powerful constraint, better than a VARCHAR , as it prevents invalid data (e.g., "admin"). DEFAULT 'Maintainer' follows the principle of least privilege.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	An audit timestamp of when the user account was created. TIMESTAMP is used over DATE to get specific time.

Example Queries:

- `SELECT * FROM Users WHERE role = 'Admin';`
- `SELECT full_name, email FROM Users WHERE user_id = 3;`

3.3.2. Projects

- **Purpose:** This table is the "what we care about." It lists the internal or external open-source projects whose dependencies are being tracked.
- **Auditor's View:** An auditor would query this to see the full "attack surface" of tracked assets and who is responsible for them.

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
project_id	INT	AUTO_INCREMENT , PRIMARY KEY	Unique numeric identifier for the project. Used as the Foreign Key.
project_name	VARCHAR(100)	NOT NULL	The common name of the project (e.g., 'Django'). NOT NULL as the project must have a name.
repo_url	VARCHAR(255)	NULL	The URL for the project's source code repository. VARCHAR(255) is a standard length for URLs. NULL is allowed because not all tracked projects may have a repo (e.g., internal-only projects, compiled binaries).
maintainer_id	INT	FOREIGN KEY (Users)	The user_id of the person responsible for this project. This is the link that creates accountability.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the project was added to the DB for tracking.

Example Queries:

- `SELECT * FROM Projects WHERE maintainer_id = 3; (Show all projects for Amit Shah).`
- `SELECT project_name, repo_url FROM Projects WHERE project_name = 'Flask';`

3.3.3. Packages

- **Purpose:** This table is a master list of all unique open-source packages (dependencies) identified across all projects. This is a core "entity" table.
- **Auditor's View:** An auditor would use this table to see which package ecosystems (npm , PyPI) are most common, indicating areas of potential systemic risk.

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
package_id	INT	AUTO_INCREMENT , PRIMARY KEY	Unique numeric identifier for the package. Used as the Foreign Key.
package_name	VARCHAR(100)	NOT NULL	The package name (e.g., 'NumPy', 'Express'). NOT NULL as the package must have a name.
ecosystem	ENUM(...)	DEFAULT 'Other'	The package manager/ecosystem: 'npm', 'PyPI', etc. ENUM is critical for filtering (e.g., "Show me all 'npm' packages").
latest_version	VARCHAR(50)	NULL	The latest known "good" or "current" version. Stored as VARCHAR as versions (e.g., 1.2.3-beta.1) are not numeric. NULL as it may not be known.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the package was first seen/added to the DB.

Example Queries:

- SELECT * FROM Packages WHERE ecosystem = 'npm';
- SELECT package_name, latest_version FROM Packages WHERE package_id = 1;

3.3.4. Vulnerabilities

- **Purpose:** This is the master list of all known security vulnerabilities (e.g., CVEs). Each vulnerability is tied to a single package.
- **Auditor's View:** This is a primary table for auditors, used to assess the known risk landscape.

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
vuln_id	INT	AUTO_INCREMENT , PRIMARY KEY	Unique numeric identifier for the vulnerability. Used as the Foreign Key.
cve_id	VARCHAR(50)	UNIQUE , NULL	The public identifier (e.g., 'CVE-2024-23334'). UNIQUE prevents duplicate CVEs. NULL is allowed for non-CVE issues (e.g., internal findings, GHSA).
package_id	INT	FOREIGN KEY (Packages)	The one package that this vulnerability affects. This is the core link.
summary	TEXT	NULL	A brief, human-readable description of the vulnerability. TEXT is used for variable,

Column Name	Data Type	Constraints	Description/Purpose
			long-form descriptions.
severity	ENUM(...)	NULL	Controlled list: 'Low', 'Medium', 'High', 'Critical'. <code>ENUM</code> is essential for consistent reporting and filtering.
cvss_score	DECIMAL(3,1)	NULL	The CVSS score (e.g., 9.8). <code>DECIMAL(3,1)</code> allows 0.0 to 10.0. <code>NULL</code> as not all vulns have a score.
published_date	DATE	NULL	The date the vulnerability was publicly disclosed. Useful for "Time to Remediate" metrics.
status	ENUM(...)	DEFAULT 'Open'	The <i>global</i> status of the vulnerability: 'Open', 'In Progress', 'Resolved'. 'Resolved' means a patch is available from the vendor.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of when this vulnerability was added to the DB.

Example Queries:

- `SELECT * FROM Vulnerabilities WHERE severity = 'Critical' AND status = 'Open';`
- `SELECT cve_id, summary, published_date FROM Vulnerabilities WHERE package_id = 1;`
(Show vulns for Werkzeug).

3.3.5. Project_Packages (Junction Table)

- **Purpose:** This is the *most important table* in the schema. It resolves the M:N relationship between `Projects` and `Packages`. It is the "Software Bill of Materials" (SBOM) table.
- **Auditor's View:** This table is the "crown jewels" for an auditor, as it maps assets (`Projects`) to their components (`Packages`) and versions.

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
projpkg_id	INT	AUTO_INCREMENT , PRIMARY KEY	Unique identifier for this <i>specific link</i> .
project_id	INT	FOREIGN KEY (Projects)	The project in the relationship (e.g., 'Flask').
package_id	INT	FOREIGN KEY (Packages)	The package in the relationship (e.g., 'Werkzeug').
version_used	VARCHAR(50)	NULL	Crucial field: The specific version of the package used by this project (e.g., '2.2.2'). This is what determines if it's vulnerable.
last_scanned	DATE	NULL	The date a scanner last verified this dependency. Essential for data freshness and staleness reports.

Example Queries:

- `SELECT * FROM Project_Packages WHERE project_id = 1;` (Show all packages used by Flask).

- `SELECT project_id FROM Project_Packages WHERE package_id = 8;` (Show all projects using Lodash).

3.3.6. Remediations

- **Purpose:** This table tracks the *action* taken to fix a vulnerability *for a specific project*. While `Vulnerabilities` tracks the *problem*, `Remediations` tracks the *solution*.
- **Auditor's View:** This table provides the *evidence* of remediation. An auditor will compare this table against `Vulnerabilities` to find gaps (e.g., "Why is this 'Critical' vuln still 'Pending?'").

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
<code>remediation_id</code>	<code>INT</code>	<code>AUTO_INCREMENT</code> , <code>PRIMARY KEY</code>	Unique identifier for this remediation effort.
<code>project_id</code>	<code>INT</code>	<code>FOREIGN KEY</code> (<code>Projects</code>)	The specific project that was/is being fixed.
<code>vuln_id</code>	<code>INT</code>	<code>FOREIGN KEY</code> (<code>Vulnerabilities</code>)	The specific vulnerability that was/is being fixed.
<code>fixed_version</code>	<code>VARCHAR(50)</code>	<code>NULL</code>	The package version the project was upgraded <i>to</i> (e.g., '3.0.1'). <code>NULL</code> if not yet fixed.
<code>status</code>	<code>ENUM(...)</code>	<code>DEFAULT 'Pending'</code>	The status of <i>this specific fix</i> : 'Pending', 'In Progress', 'Resolved'. This is the <i>local</i> status, distinct from the global <code>Vulnerabilities.status</code> .
<code>date_applied</code>	<code>DATE</code>	<code>NULL</code>	The date the fix was deployed to production. This is the key metric for "Time to Remediate."
<code>created_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT CURRENT_TIMESTAMP</code>	Timestamp of when this remediation ticket was "opened" or first logged.

Example Queries:

- `SELECT * FROM Remediations WHERE status = 'Pending';`
- `SELECT * FROM Remediations WHERE project_id = 1 AND status = 'Resolved';` (Show resolved items for Flask).

3.3.7. Audit_Log

- **Purpose:** A simple, high-level log to track user actions for accountability and compliance. It answers "Who did what, and when?"
- **Auditor's View:** This is the "paper trail." An auditor will use this to verify actions, such as "Who marked this vulnerability as 'Resolved'?"

Column Breakdown:

Column Name	Data Type	Constraints	Description/Purpose
log_id	INT	AUTO_INCREMENT , PRIMARY KEY	Unique identifier for the log entry.
user_id	INT	FOREIGN KEY (Users)	The user who performed the action. Links the "what" to the "who."
action	VARCHAR(255)	NULL	A short, machine-readable description of the action (e.g., 'Added vulnerability', 'Updated remediation').
details	TEXT	NULL	More context (e.g., "Set CVE-2024-29018 to In Progress for project ExpressJS").
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	The exact time the action occurred. This is the most critical field.

Example Queries:

- `SELECT * FROM Audit_Log WHERE user_id = 1 ORDER BY timestamp DESC;` (Show all actions by Ravi Kumar).
- `SELECT * FROM Audit_Log WHERE action = 'Updated remediation';`

4. Data Model & Sample Data

4.1. Rationale for Sample Data

The provided `INSERT` statements are the most valuable part of this project for a learner. A schema is just a blueprint; sample data builds the house. This data was curated to be:

- **Realistic:** Uses real, well-known projects (Flask, React) and packages (Werkzeug, Lodash). This provides immediate context and makes the schema feel tangible and relevant.
- **Coherent:** The foreign keys all link up to tell a logical story. You can run complex `JOIN` queries and get results immediately. This is critical for avoiding the "empty result set" frustration that often plagues new database projects.
- **Complex:** It models M:N relationships, 1:N relationships, and provides a mix of statuses (`Open` , `Resolved` , `Pending`). This ensures that queries are interesting and reflective of a real-world, in-progress state, not a sterile, "perfect" dataset.
- **Traceable:** The data is simple enough (10 rows per table) that a user can manually trace the relationships, as shown below. This "manual" tracing is a powerful way to understand how `JOIN` s work under the hood.

4.2. Data Coherency & Storytelling

The schema's power is shown by tracing data through the tables.

Scenario 1: The "Flask / Werkzeug" Vulnerability (Active & Resolved)

This story shows how a vulnerability is identified, linked to a project, and resolved.

Step	Table	Record (Key Data)	Explanation
1	Projects	project_id: 1 , project_name: 'Flask'	We are tracking the 'Flask' project.

Step	Table	Record (Key Data)	Explanation
2	Packages	package_id: 1 , package_name: 'Werkzeug'	We know a package 'Werkzeug' exists.
3	Project_Packages	project_id: 1 , package_id: 1 , version_used: '2.2.2'	The key link! We now know 'Flask' uses 'Werkzeug' v2.2.2.
4	Vulnerabilities	vuln_id: 1 , cve_id: 'CVE-2024-23334' , package_id: 1	We discover a CVE that affects 'Werkzeug' (package_id 1).
5	(Analysis)	summary: '...in Werkzeug < 3.0.0'	Our scanner (or an analyst) compares version_used (2.2.2) to the summary (< 3.0.0) and confirms Flask is vulnerable .
6	Remediations	project_id: 1 , vuln_id: 1 , status: 'Resolved' , fixed_version: '3.0.1'	The solution! We have a record showing the 'Flask' project <i>fixed</i> this <i>specific vulnerability</i> by upgrading to '3.0.1' on '2025-02-01'.
7	Audit_Log	log_id: 1 , user_id: 1 , action: 'Added vulnerability'	We can even see <i>who</i> (Ravi Kumar, user 1) added the vulnerability to the database.

Scenario 2: The "React / Lodash" Vulnerability (Pending)

This story shows a vulnerability that is *known* but not yet *fixed* by all projects.

Step	Table	Record (Key Data)	Explanation
1	Projects	project_id: 8 , project_name: 'Angular'	We are tracking the 'Angular' project.
2	Packages	package_id: 8 , package_name: 'Lodash'	We know the 'Lodash' package exists.
3	Project_Packages	project_id: 8 , package_id: 8 , version_used: '4.17.20'	Link 1: 'Angular' uses 'Lodash' v4.17.20.
4	Vulnerabilities	vuln_id: 9 , cve_id: 'CVE-2024-16002' , package_id: 8	We discover a CVE that affects 'Lodash' (package_id 8).
5	(Analysis)	(Summary implies < 4.17.21 is bad)	An analyst compares version_used (4.17.20) and confirms Angular is vulnerable .
6	Remediations	remediation_id: 9 , project_id: 8 , vuln_id: 9 , status: 'Resolved'	The solution! The 'Angular' project (ID 8) has <i>fixed</i> this vulnerability (ID 9).
7	Projects	project_id: 2 , project_name: 'React'	Now, let's look at a <i>different</i> project.
8	Project_Packages	<i>No record for (project_id: 2, package_id: 8)</i>	Let's <i>assume</i> React also uses Lodash. A scanner would add a Project_Packages row.
9	Remediations	remediation_id: 2 , project_id: 2 , vuln_id: 2 , status: 'Pending'	This is a <i>different</i> vulnerability, but it shows that the 'React' project (ID 2) has a 'Pending' item, demonstrating its queue.

This traceability is essential for making the database a useful tool for learning and development.

4.3. Sample Data Walkthrough

- **Users** : Contains 10 users with a mix of 'Admin' (Ravi, Priya, Sophia), 'Analyst' (Neha, John, Olivia), and 'Maintainer' (Amit, Sarah, Mark, Liam) roles. This allows for testing role-based logic.
- **Projects** : Contains 10 popular OSS projects, assigning them to different `maintainer_id` s. Note that 'Amit Shah' (ID 3) maintains *two* projects (Flask, TensorFlow), demonstrating a 1:N relationship.
- **Packages** : Contains 10 common packages that logically map to the projects.
- **Vulnerabilities** : Contains 10 realistic CVEs, each mapped to a `package_id` and given a `severity` , `cvss_score` , and `status` . Note the mix of 'Critical' and 'High' severities.
- **Project_Packages** : Contains 10 "links." Crucially, `project_id: 1` (Flask) has two entries, showing it uses both `package_id: 1` (Werkzeug) and `package_id: 2` (Jinja2). This models a real project's dependency tree.
- **Remediations** : Contains 10 records tracking the *local* fix status. The `status` column is a mix of 'Resolved', 'In Progress', and 'Pending', which makes `vw_remediation_progress` immediately useful.
- **Audit_Log** : Contains 10 sample log entries, linking various `user_id` s to various actions, providing a realistic audit trail.

5. Database Views (The Analytics Layer)

5.1. Purpose of the View Layer

The `VIEW` s are not strictly necessary for the database to *function*, but they are *essential* for making it *usable*. They serve three main purposes:

1. **Abstraction & Simplicity:** They hide the complexity of `JOIN` s. A user doesn't need to be a SQL expert to get answers.
2. **Security & Access Control:** In a production system, you can `GRANT SELECT` permission to an analyst (Persona: "Neha") on `vw_project_vulnerabilities` *without* giving her `SELECT` (or `UPDATE` / `DELETE`) permissions on the underlying tables. This limits her access to only the data she needs, in the format she needs it.
3. **Consistency & Maintainability:** They ensure that everyone who asks the same question (e.g., "What is our project risk?") gets the same answer, because the join logic and calculations are defined in *one place*. If the logic needs to change, you update the `VIEW` once, and all reports that use it are instantly updated.

5.2. View-by-View Analysis

This section provides a detailed technical breakdown of each view.

5.2.1. `vw_all_vulnerabilities`

- **Stakeholder Question:** "Just give me a master list of all the vulnerabilities we are tracking, and what package they belong to."
- **Audience:** Security Analysts (Persona: "Neha").
- **SQL Breakdown:** This is a simple and fast view that joins the two primary "problem" entities: `Vulnerabilities` and `Packages` . It uses an `INNER JOIN` on `v.package_id = p.package_id` . This means that if a vulnerability were somehow entered without a valid `package_id` (which the foreign key constraint *should* prevent), it would not appear in this list. Its main purpose is to de-normalize the data, translating the `package_id` (e.g., `1`) into a human-readable `package_name` (e.g., 'Werkzeug').

Column Breakdown:

Column Name	Source Table	Source Column	Data Type	Description
vuln_id	Vulnerabilities	vuln_id	INT	The unique primary key for the vulnerability.
cve_id	Vulnerabilities	cve_id	VARCHAR(50)	The public identifier for the vulnerability (e.g., 'CVE-2024-23334').
package_name	Packages	package_name	VARCHAR(100)	(De-normalized) The human-readable name of the package this vulnerability affects.
severity	Vulnerabilities	severity	ENUM(...)	The assigned severity ('Low', 'Medium', 'High', 'Critical').
status	Vulnerabilities	status	ENUM(...)	The <i>global</i> status of the vulnerability ('Open', 'In Progress', 'Resolved').
published_date	Vulnerabilities	published_date	DATE	The date the vulnerability was publicly disclosed.

- **Sample Query:** `SELECT * FROM vw_all_vulnerabilities WHERE severity = 'Critical' AND status = 'Open';`

5.2.2. vw_project_vulnerabilities

- **Stakeholder Question:** "What is the *actual risk* to my projects? Show me every vulnerability in every project I own and tell me if we've fixed it yet."
- **Audience:** Project Maintainers (Persona: "Amit"), Security Managers (Persona: "Priya").
- **SQL Breakdown:** This is the most complex and important view, acting as the primary risk register. It performs a "chain join" across four tables to connect projects to the vulnerabilities they are exposed to: `Projects -> Project_Packages -> Packages -> Vulnerabilities`.
 1. It joins `Projects` (`pr`) with `Project_Packages` (`pp`) to find *which* packages a project uses.
 2. It joins that result with `Packages` (`pk`) to get the package *names*.
 3. It joins that result with `Vulnerabilities` (`v`) to find all vulnerabilities associated with those packages.
 4. Finally, it uses a crucial `LEFT JOIN` on `Remediations` (`r`). This is essential because it includes *all* vulnerabilities, even those that *do not* have a `Remediations` record yet.
 5. The `COALESCE(r.status, 'Pending')` logic states: "Show the remediation status if it exists. If it's `NULL` (meaning no record exists), default to showing 'Pending'."

Column Breakdown:

Column Name	Source Table	Source Column	Data Type	Description
project_name	Projects	project_name	VARCHAR(100)	The name of the tracked project.
package_name	Packages	package_name	VARCHAR(100)	The name of the package this project uses.

Column Name	Source Table	Source Column	Data Type	Description
version_used	Project_Packages	version_used	VARCHAR(50)	The <i>specific version</i> of the package in use by this project.
cve_id	Vulnerabilities	cve_id	VARCHAR(50)	The public identifier for the vulnerability affecting this package.
severity	Vulnerabilities	severity	ENUM(...)	The severity of the vulnerability.
summary	Vulnerabilities	summary	TEXT	The description of the vulnerability.
remediation_status	Remediations / COALESCE	status	ENUM(...)	(Calculated) The <i>local</i> fix status ('Pending', 'In Progress', 'Resolved') for this specific project and vulnerability. Defaults to 'Pending' if no remediation record exists.

- **Sample Query:** `SELECT * FROM vw_project_vulnerabilities WHERE project_name = 'Flask' AND remediation_status != 'Resolved';`

5.2.3. vw_package_vuln_count

- **Stakeholder Question:** "Which of our packages are the *most* problematic? Where should we focus our efforts for replacement or library-wide patching?"
- **Audience:** Security Analysts (Persona: "Neha"), Architects.
- **SQL Breakdown:** This is an aggregate query. It uses a `LEFT JOIN` from `Packages (pk)` to `Vulnerabilities (v)`. The `LEFT JOIN` is important so that packages with *zero* vulnerabilities are still included in the list (with a count of 0). It then uses `GROUP BY pk.package_name` to collapse all rows for a single package (e.g., 'Lodash') into one row, and `COUNT(v.vuln_id)` to count how many vulnerabilities were associated with it.

Column Breakdown:

Column Name	Source Table	Source Column	Data Type	Description
package_name	Packages	package_name	VARCHAR(100)	The name of the package. This is the <code>GROUP BY</code> key.
vulnerability_count	Vulnerabilities	COUNT(vuln_id)	INT	(Aggregated) The total number of vulnerabilities associated with this package.

- **Sample Query:** `SELECT * FROM vw_package_vuln_count ORDER BY vulnerability_count DESC LIMIT 10;`

5.2.4. vw_project_package_usage

- **Stakeholder Question:** "What versions of what packages are our projects *actually* using? I need our 'Software Bill of Materials' (SBOM)."
- **Audience:** Developers (Persona: "Amit"), Auditors.
- **SQL Breakdown:** This is a simple "bill of materials" view. It `JOIN`s `Projects`, `Project_Packages`, and `Packages` to create a simple, human-readable list of dependencies. It's the primary way to see the M:N relationship between projects and packages.

Column Breakdown:

Column Name	Source Table	Source Column	Data Type	Description
project_name	Projects	project_name	VARCHAR(100)	The name of the tracked project.
package_name	Packages	package_name	VARCHAR(100)	The name of the package this project uses.
version_used	Project_Packages	version_used	VARCHAR(50)	The specific version of the package in use by this project.
last_scanned	Project_Packages	last_scanned	DATE	The date this dependency link was last verified by a scanner.

- **Sample Query:** `SELECT * FROM vw_project_package_usage WHERE package_name = 'Lodash';`
(Shows all projects using Lodash).

5.2.5. vw_critical_vulnerabilities

- **Stakeholder Question:** "Emergency! Show me *only* the 'Critical' vulnerabilities right now. I don't care about 'Medium' or 'Low'."
- **Audience:** Security Managers (Persona: "Priya"), Admins.
- **SQL Breakdown:** This view is almost identical to `vw_all_vulnerabilities` (it joins `Vulnerabilities` and `Packages`), but it has a non-negotiable `WHERE v.severity = 'Critical'` clause built directly into the view definition. This provides a "filtered list" for high-priority dashboards and ensures that consumers of this view *cannot* accidentally see lower-severity issues.

Column Breakdown:

Column Name	Source Table	Source Column	Data Type	Description
cve_id	Vulnerabilities	cve_id	VARCHAR(50)	The public identifier for the vulnerability.
package_name	Packages	package_name	VARCHAR(100)	(De-normalized) The name of the package this <i>critical</i> vulnerability affects.
summary	Vulnerabilities	summary	TEXT	The description of the vulnerability.
severity	Vulnerabilities	severity	ENUM(...)	The severity, which will <i>always</i> be 'Critical' for this view.
status	Vulnerabilities	status	ENUM(...)	The <i>global</i> status of the vulnerability ('Open', 'In

Column Name	Source Table	Source Column	Data Type	Description
				Progress', 'Resolved').

- **Sample Query:** `SELECT * FROM vw_critical_vulnerabilities WHERE status = 'Open';`

5.2.6. vw_remediation_progress

- **Stakeholder Question:** "How are we doing? Give me the 10,000-foot view of our fix-it funnel for my management dashboard."
- **Audience:** C-Suite, Security Managers (Persona: "Priya").
- **SQL Breakdown:** This is a "dashboard widget" query. It reads *only* from the `Remediations` table. It performs no `JOIN`s and has no `WHERE` clause, so it scans the entire table. It uses conditional aggregation (`SUM(status='...')`) to pivot the data. `SUM(status='Resolved')` works because in MySQL, a true condition (`status='Resolved'`) evaluates to `1` and a false one to `0`. This query collapses all rows into a *single row* with three columns, perfect for a BI dashboard.

Column Breakdown:

Column Name	Source Table	Source Column	Data Type	Description
<code>resolved_total</code>	<code>Remediations</code>	<code>SUM(status='Resolved')</code>	<code>INT</code>	(Aggregated) The total count of all remediation records marked as 'Resolved'.
<code>inprogress_total</code>	<code>Remediations</code>	<code>SUM(status='In Progress')</code>	<code>INT</code>	(Aggregated) The total count of all remediation records marked as 'In Progress'.
<code>pending_total</code>	<code>Remediations</code>	<code>SUM(status='Pending')</code>	<code>INT</code>	(Aggregated) The total count of all remediation records marked as 'Pending'.

- **Sample Query:** `SELECT * FROM vw_remediation_progress;`

6. Stored Procedures (The Operational Layer)

6.1. Purpose of Stored Procedures

If `VIEW`s are for *reading* data, `PROCEDURE`s are for *writing* it. They provide a safe, controlled, and auditable "API" for the database.

1. **Encapsulation (Black Box):** The application doesn't need to know the schema details or the business rules. It just `CALL`s a procedure with parameters. If the business rule changes (e.g., `date_applied` should be `NOW()` instead of `CURDATE()`), you change the procedure, and *no application code needs to be re-deployed*.
2. **Security (Preventing SQL Injection):** This is the most critical reason. By using procedures, the application provides data as *parameters*, not as part of a concatenated SQL string. The database engine handles the data safely, preventing malicious input (like `'OR 1=1; --`) from destroying the query.

3. **Performance:** Stored procedures can have their execution plans compiled and cached by the database engine, which can lead to better performance for frequently executed, complex write operations.

6.2. Procedure Analysis

This section provides a detailed technical breakdown of each procedure and function.

6.2.1. UpdateVulnerabilityStatus

- **Signature:** `PROCEDURE UpdateVulnerabilityStatus(IN p_vuln_id INT, IN p_status ENUM('Open', 'In Progress', 'Resolved'))`
- **Purpose:** To update the *global* status of a single vulnerability in the `Vulnerabilities` table.
- **Business Logic:** This procedure is used to reflect the *global* state of a vulnerability, not a *local* fix. For example, when a vendor (like Apache) releases a patch for a vulnerability, an Admin or Analyst would call this procedure to change the vulnerability's status from 'Open' to 'Resolved', indicating a fix is *available*.

Parameter Breakdown:

Parameter	Type	IN / OUT	Description
p_vuln_id	INT	IN	The Primary Key (<code>vuln_id</code>) of the vulnerability to be updated.
p_status	ENUM(...)	IN	The new status to set. The <code>ENUM</code> type provides inherent validation.

Logic Breakdown:

1. The procedure receives `p_vuln_id` and `p_status`.
 2. It executes a single `UPDATE` statement: `UPDATE Vulnerabilities SET status = p_status WHERE vuln_id = p_vuln_id;`
 3. This is a direct, targeted update on a single row based on its primary key.
- **Example Call:** `CALL UpdateVulnerabilityStatus(4, 'Resolved');` (Marks `CVE-2024-29018` as globally resolved).

6.2.2. AddRemediation

- **Signature:** `PROCEDURE AddRemediation(IN p_project INT, IN p_vuln INT, IN p_version VARCHAR(50), IN p_status ENUM('Pending', 'In Progress', 'Resolved'))`
- **Purpose:** To create a new *project-specific* remediation "ticket" in the `Remediations` table.
- **Business Logic:** This is the primary "action" procedure. It's called when a vulnerability is first discovered for a project (to create a 'Pending' record) or when a developer has applied a fix (to create a 'Resolved' record). Its key business rule is setting the `date_applied` to `CURDATE()`, which timestamps the fix *at the time of logging* and ensures this date cannot be tampered with by the client application.

Parameter Breakdown:

Parameter	Type	IN / OUT	Description
p_project	INT	IN	The <code>project_id</code> for this remediation (e.g., <code>1</code> for 'Flask').
p_vuln	INT	IN	The <code>vuln_id</code> that is being fixed (e.g., <code>1</code> for 'CVE-2024-23334').
p_version	VARCHAR(50)	IN	The new, <i>fixed</i> version the project was upgraded to (e.g., '3.0.1'). Can be <code>NULL</code> if status is 'Pending'.

Parameter	Type	IN / OUT	Description
p_status	ENUM(...)	IN	The new <i>local</i> status for this fix (e.g., 'Resolved').

Logic Breakdown:

1. The procedure receives the four `IN` parameters.
 2. It executes a single `INSERT` statement into the `Remediations` table.
 3. It explicitly maps the parameters to their columns and sets `date_applied = CURDATE()`.
- **Example Call:** `CALL AddRemediation(1, 2, '3.0.1', 'In Progress');` (Creates a new 'In Progress' ticket for project 1, vuln 2).

6.2.3. AddVulnerabilitySimple

- **Signature:** `PROCEDURE AddVulnerabilitySimple(IN p_cve VARCHAR(50), IN p_package_id INT, IN p_summary TEXT, IN p_severity ENUM('Low', 'Medium', 'High', 'Critical'), IN p_cvss DECIMAL(3,1))`
- **Purpose:** A simple helper procedure to add a new *global* vulnerability to the `Vulnerabilities` table.
- **Business Logic:** This procedure simplifies the `INSERT` process for a client application. It sets the `published_date` to `CURDATE()` by default, assuming the vulnerability is being logged on the day it was discovered/published. This is a common workflow for analysts.

Parameter Breakdown:

Parameter	Type	IN / OUT	Description
p_cve	VARCHAR(50)	IN	The CVE identifier (e.g., 'CVE-2025-0001').
p_package_id	INT	IN	The <code>package_id</code> this vulnerability is associated with.
p_summary	TEXT	IN	The text description of the vulnerability.
p_severity	ENUM(...)	IN	The assigned severity.
p_cvss	DECIMAL(3,1)	IN	The CVSS score (e.g., 7.5).

Logic Breakdown:

1. The procedure receives the five `IN` parameters.
 2. It executes a single `INSERT` statement into the `Vulnerabilities` table.
 3. It maps the parameters to their columns and sets `published_date = CURDATE()`.
- **Example Call:** `CALL AddVulnerabilitySimple('CVE-2025-1000', 1, 'New Test Vuln', 'High', 7.5);`

6.2.4. CountVulnsForPackage (Function)

- **Signature:** `FUNCTION CountVulnsForPackage(p_package INT) RETURNS INT DETERMINISTIC`
- **Purpose:** A simple utility function to get the total number of vulnerabilities for a single package.
- **Business Logic:** This demonstrates a `FUNCTION` instead of a `PROCEDURE`. A function can be used directly within a `SELECT` statement, which is very useful. It is marked `DETERMINISTIC` because it will *always* return the same result for the same input *within a single transaction* (this is a hint for the query optimizer).

Parameter Breakdown:

Parameter	Type	IN / OUT	Description
p_package	INT	IN	The <code>package_id</code> you want to count vulnerabilities for.

Logic Breakdown:

1. The function receives `p_package`.
 2. It declares a local variable `total` of type `INT`.
 3. It runs `SELECT COUNT(*) INTO total FROM Vulnerabilities WHERE package_id = p_package;`.
This counts the rows in `Vulnerabilities` matching the package ID and stores the result in the `total` variable.
 4. It `RETURN`s the `total` variable.
- **Example Call:** `SELECT package_name, CountVulnsForPackage(package_id) AS total_vulns FROM Packages WHERE package_name = 'Werkzeug';`

7. Example Workflows & Use Cases

7.1. Use Case 1: Daily Scan & Vulnerability Ingestion

Goal: A new scan runs, identifies a new vulnerability in a project, and ingests it. **Actor:** Automated System or Security Analyst (Neha)

1. **Scan:** An external tool (like `trivy` or `snyk-cli`) scans the 'React' (`project_id` 2) project.
2. **Identify:** It finds 'React' uses 'Lodash' (`package_id` 8) at `version_used`: `'4.17.20'`.
3. **Check/Update DB Link:** The system upserts this information.
 - `INSERT INTO Project_Packages (project_id, package_id, version_used, last_scanned) VALUES (2, 8, '4.17.20', CURDATE()) ON DUPLICATE KEY UPDATE version_used = '4.17.20', last_scanned = CURDATE();`
4. **Find Vuln:** The scanner reports a *new* CVE, 'CVE-2025-0001', for 'Lodash', affecting versions `< 4.17.21`.
5. **Check/Insert Vuln:** The system checks if this CVE is known.
 - `SELECT * FROM Vulnerabilities WHERE cve_id = 'CVE-2025-0001';`
 - It returns empty. The system inserts it using the procedure:
 - `CALL AddVulnerabilitySimple('CVE-2025-0001', 8, 'Lodash prototype pollution v2', 'High', 8.2);`
 - (Let's assume this new `vuln_id` is 11).
6. **Create Remediation Task:** This is the key step. The system must link the *project* to the *vulnerability*. It checks if a remediation record already exists.
 - `SELECT * FROM Remediations WHERE project_id = 2 AND vuln_id = 11;`
 - It returns empty. It creates a new 'Pending' task:
 - `CALL AddRemediation(2, 11, NULL, 'Pending');`
7. **Result:** A new 'Pending' remediation task is now in the system for the React project. A manager (Priya) running `SELECT * FROM vw_project_vulnerabilities WHERE remediation_status = 'Pending';` will now see this new item.

7.2. Use Case 2: Patch Application & Remediation Lifecycle

Goal: A developer fixes a "Pending" vulnerability. **Actor:** Project Maintainer (Sarah Lee, ID 4)

1. **Identify Tasks:** Sarah queries her work queue (she maintains 'React', ID 2).
 - `SELECT * FROM vw_project_vulnerabilities WHERE project_name = 'React' AND remediation_status = 'Pending';`
 - She sees the new item: CVE-2025-0001 for 'Lodash', version_used: 4.17.20.
2. **Develop:** She sees the fix is to upgrade to 4.17.21. She updates the `package.json` and opens a Pull Request. She (or an automated system) updates the remediation ticket to show work has started.
 - `UPDATE Remediations SET status = 'In Progress' WHERE project_id = 2 AND vuln_id = 11;`
3. **Audit (Automatic):** An application trigger logs this change.
 - `INSERT INTO Audit_Log (user_id, action, details) VALUES (4, 'Updated remediation', 'Set CVE-2025-0001 for React to In Progress');`
4. **Deploy:** The PR is merged and deployed to production.
5. **Close Task:** A CI/CD pipeline job confirms the fix and calls the database:
 - `UPDATE Remediations SET status = 'Resolved', fixed_version = '4.17.21', date_applied = CURDATE() WHERE project_id = 2 AND vuln_id = 11;`
6. **Audit (Automatic):**
 - `INSERT INTO Audit_Log (user_id, action, details) VALUES (4, 'Resolved remediation', 'Set CVE-2025-0001 for React to Resolved, v4.17.21');`
7. **Result:** The vulnerability no longer shows up as "Pending" for the React project. The `vw_remediation_progress` view now shows one *less* 'Pending' and one *more* 'Resolved' item.

7.3. Use Case 3: Management Reporting & Risk Analysis

Goal: An Admin or Manager needs a high-level report on the organization's security posture. **Actor:** Admin (Priya, ID 6) or Analyst (Neha, ID 2)

1. **Dashboard View:** Priya opens her BI dashboard, which runs this query:
 - `SELECT * FROM vw_remediation_progress;`
 - **Result:** resolved_total: 5, inprogress_total: 1, pending_total: 4 (Based on sample data). She sees 4 pending items and asks Neha to investigate.
2. **Worst Offenders (Packages):** Neha wants to know which packages cause the most "noise" or risk.
 - `SELECT * FROM vw_package_vuln_count ORDER BY vulnerability_count DESC;`
 - **Result:** Shows that most packages have 1 vulnerability. If 'Lodash' had 5, it would be the clear outlier.
3. **Worst Offenders (Projects/Maintainers):** Neha needs to find the "riskiest" projects or maintainers with the most 'Pending' high-priority tasks.
 - `SELECT pr.project_name, u.full_name as maintainer, COUNT(v.vuln_id) as pending_vulns FROM Remediations r JOIN Projects pr ON r.project_id = pr.project_id JOIN Users u ON pr.maintainer_id = u.user_id JOIN Vulnerabilities v ON r.vuln_id = v.vuln_id WHERE r.status = 'Pending' AND v.severity IN ('Critical', 'High') GROUP BY pr.project_name, u.full_name ORDER BY pending_vulns DESC;`
 - **Result:** A ranked list of projects and their owners who need to be contacted.
4. **Immediate Threat Triage:** An urgent alert comes in for 'CVE-2024-23155' (the 'Pandas' RCE). Priya needs to know "Are we affected?"
 - `SELECT * FROM vw_project_vulnerabilities WHERE cve_id = 'CVE-2024-23155';`
 - **Result:** project_name: 'Pandas', package_name: 'Pandas', version_used: '2.1.5', ..., remediation_status: 'Pending'.

- **Answer:** "Yes, our 'Pandas' project, maintained by Neha Patel, is vulnerable and the fix is still pending." She can now act on this specific, qualified information.

8. Deployment & Operations

8.1. System Requirements

- **Database:** MySQL Server 8.0 or higher (for `ENUM` and `TIMESTAMP` support).
- **Client:** Any standard SQL client (MySQL Workbench, DBeaver, `mysql CLI`).
- **Permissions:** A user with `CREATE DATABASE`, `CREATE TABLE`, `CREATE VIEW`, `CREATE PROCEDURE`, `INSERT`, and `SELECT` privileges.

8.2. Installation & Setup

A single, runnable SQL script (`vmdb_oss_full_script.sql`) is provided. This script is **idempotent**, meaning it will safely `DROP` and `CREATE` the database, allowing it to be run multiple times without error.

Steps:

1. Save the contents of the `.sql` file provided.
2. Execute the script using a MySQL client.
 - **Via CLI:** `mysql -u [username] -p < vmdb_oss_full_script.sql`
 - **Via Workbench:** Open the `.sql` file and click the "Execute" (lightning bolt) icon.

8.3. Verification & Basic Usage

Run the following queries to confirm the data was created and the views are working:

1. **Check Users:**
 - `USE VMDB_OSS;`
 - `SELECT * FROM Users LIMIT 1;`
 - *Expected:* Shows 'Ravi Kumar'.
2. **Check Package Counts:**
 - `SELECT * FROM vw_package_vuln_count;`
 - *Expected:* A list of 10 packages and their vulnerability counts (mostly 1).
3. **Check Project Risk:**
 - `SELECT * FROM vw_project_vulnerabilities WHERE project_name = 'Flask';`
 - *Expected:* Shows the 2 vulnerabilities affecting Flask (from Werkzeug and Jinja2) and their remediation statuses.
4. **Check Dashboard View:**
 - `SELECT * FROM vw_remediation_progress;`
 - *Expected:* A single row with `resolved_total`, `inprogress_total`, `pending_total`.
5. **Test Foreign Key (Should Fail):**
 - `INSERT INTO Remediations (project_id, vuln_id, status) VALUES (999, 999, 'Pending');`
 - *Expected:* An error, e.g., `Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails...` This *proves* the schema integrity is working.

9. Project Deliverables & Evaluation

9.1. Core Artifacts

This project consists of two primary artifacts:

1. `VMDB_OSS_Design_Doc.md` (This file): A comprehensive Markdown document explaining the project's purpose, design, and use.
2. `vmdb_oss_full_script.sql`: A single, runnable SQL file that contains the complete DDL (Data Definition Language) and DML (Data Manipulation Language) to create and populate the entire database, including tables, views, and procedures.

9.2. Demonstration Points

To evaluate or demonstrate this project, an instructor or reviewer should follow this "script":

1. **Show the Code:** Briefly show the `vmdb_oss_full_script.sql` file, pointing out the 7 tables, 6 views, and 3 procedures/functions.
2. **Run the Code:** Execute the full script in MySQL Workbench.
3. **Show Basic Data:** Run `USE VMDB_OSS;` and `SELECT * FROM Users;` to prove data was inserted.
4. **Show the "Problem" (The Raw Data):**
 - Run `SELECT * FROM Project_Packages WHERE project_id = 1;` (Shows Flask uses package 1 and 2).
 - Run `SELECT * FROM Vulnerabilities WHERE package_id = 1;` (Shows package 1 has vuln 1).
 - *Explain:* "This is confusing. Now let's use the views."
5. **Show the "Solution" (The Views):**
 - Run `SELECT * FROM vw_project_vulnerabilities WHERE project_name = 'Flask';`
 - *Explain:* "See? The view automatically joins all that data to show us Flask is affected by two vulns, and tells us the fix status for both."
6. **Show the "Dashboard":**
 - Run `SELECT * FROM vw_remediation_progress;`
 - *Explain:* "This single view gives a manager their high-level dashboard."
7. **Show the "API" (The Procedure):**
 - *Before:* `SELECT * FROM Remediations WHERE project_id = 2 AND vuln_id = 2;` (Should show one 'Pending' row).
 - *Run:* `UPDATE Remediations SET status = 'In Progress' WHERE project_id = 2 AND vuln_id = 2;` (Or `CALL AddRemediation...`)
 - *After:* `SELECT * FROM Remediations WHERE project_id = 2 AND vuln_id = 2;` (Should show 'In Progress').
 - *Explain:* "We safely modified data, and this would normally be done via a procedure call."
8. **Show the "Integrity":**
 - Run the failing `INSERT` query from section 8.3 to prove the database protects itself from bad data.

10. Future Roadmap & Extensions

10.1. Short-Term Goals (Ingestion & UI)

- **Web UI:** Build a simple Flask, FastAPI, or React front-end. The main page would be a dashboard showing `vw_remediation_progress` and `vw_critical_vulnerabilities`. It would have drill-down pages to see `vw_project_vulnerabilities` and forms to call the stored procedures.

- **NVD Importer:** Write a Python script (`importer.py`) that runs daily via cron. It would `curl` the NVD JSON feed, parse it, and `INSERT` or `UPDATE` records in the `Vulnerabilities` and `Packages` tables.
- **SBOM Parsing:** Add a feature to the UI where a user can upload a `CycloneDX` or `SPDX` JSON file. The application would parse this "Software Bill of Materials" and automatically populate the `Project_Packages` table for a given `project_id`.

10.2. Medium-Term Goals (Automation & Scoring)

- **Risk Scoring:** Add a `risk_score` column to `vw_project_vulnerabilities`. This would be a calculated field (e.g., `cvss_score * (days_open / 10)`), allowing teams to prioritize not just by severity, but by *age*.
- **Automated Triggers:** Add database `TRIGGER`s. For example: `AFTER UPDATE ON Remediations`: if the `new.status` is 'Resolved', automatically `INSERT` a record into the `Audit_Log` table. This moves logic from the application into the DB.
- **GitHub Integration:** Use the GitHub API to read `dependabot.yml` or `package-lock.json` files directly from repos to populate `Project_Packages`, creating a "live" SBOM.

10.3. Long-Term Goals (Enterprise Features)

- **True RBAC:** Expand the `Users` table and add `Permissions` and `User_Groups` tables to implement Role-Based Access Control (e.g., "Maintainers (Group) can only `SELECT` from `vw_project_vulnerabilities` where `project_id` is in their group").
- **Multi-Tenant:** Add an `Organization` table. All other tables (`Projects`, `Users`, `Packages`, etc.) would get an `org_id` foreign key. This would allow a single database instance to securely serve multiple, isolated customers.
- **Docker Deployment:** Create a `docker-compose.yml` file that spins up two services: `db` (using the `mysql:8.0` image and the `.sql` script) and `app` (using the Flask/React UI from 10.1), creating a one-command "Vulnerability Management System" deployment.

11. Conclusion

Project Value: `VMDB_OSS` successfully models the complex domain of open-source vulnerability management in a simple, normalized, and relational database. It serves as a powerful testament to "getting the fundamentals right." By leveraging core database principles (3NF, Foreign Keys, Views, Procedures), it provides a solution that is simultaneously:

- **Educational:** Simple enough for a student to understand and learn from.
- **Practical:** Robust enough for a small organization to build real tools upon.
- **Trustworthy:** Guaranteed to be consistent and integral by its very design.

While simple, this project is not trivial. It is a robust, "real-world" schema that is immediately useful for learning, building internal tools, or managing small-scale security operations. The provided data, views, and procedures create a fully functional system "out-of-the-box," making it an ideal project for a portfolio or an educational course.